PetCare — Reparto de tareas de esta entrega (explicado fácil)

¿Qué vamos a construir en esta entrega?

El flujo de la app: **Home** → **Mis Mascotas** → **Detalle de Rocky** → **Registrar/Editar Cuidado**.

Los datos de Rocky y de sus cuidados (vacunas, baños, desparasitaciones, visitas al veterinario) viven en una base de datos real (Room), no en memoria — si cierras la app por completo y la vuelves a abrir, todo sigue ahí. Cada pantalla tiene su propio ViewModel, que es quien de verdad le pide los datos a la base — la pantalla (lo visual) nunca habla directo con la base de datos. La navegación entre pantallas es manual (una variable de estado en el contenedor decide qué pantalla mostrar), sin usar todavía la librería NavHost/NavController.

Las 3 tablas de la base de datos

Tabla	Tipo	Qué guarda
`Mascota`	Maestra	Los datos de cada mascota (por ahora solo Rocky, precargado). Es de solo lectura esta entrega.
`TipoCuidado`	Maestra	Catálogo fijo: Vacunación, Baño, Desparasitación, Visita veterinaria.
`Cuidado`	Transaccional	Cada vez que se registra un cuidado: qué mascota, qué tipo, qué fecha, qué notas. **Esta sí tiene CRUD completo** (crear, ver, editar, eliminar).

El flujo, pantalla por pantalla

Pantalla	Qué muestra	De dónde saca los datos
Home	Resumen: cuántas mascotas hay	`HomeViewModel` le pregunta al Repositorio cuántas hay
Mis Mascotas	Lista de mascotas (por ahora solo Rocky)	`MascotasViewModel` le pide al Repositorio la lista completa
Detalle	Datos de la mascota + su lista de cuidados	`DetalleMascotaViewModel` combina los datos de la mascota y sus cuidados
Registrar/Editar Cuidado	Formulario: tipo de cuidado, fecha, notas	`RegistrarCuidadoViewModel` — sirve para crear, editar y eliminar

Conceptos que hay que entender (para la presentación también)

Concepto	Qué es	Analogía simple
ViewModel	Clase que le pide los datos al Repositorio y los deja listos para que la pantalla los muestre. Sobrevive a rotar el celular.	El "mesero" que va a la cocina, trae el plato y te lo sirve — tú nunca entras a la cocina.
UiState	Un "sobre" con 3 posibles contenidos: `Loading`, `Success` o `Error`. La pantalla solo pregunta qué hay en el sobre ahora.	Como el estado de un pedido: "preparando", "listo" o "hubo un problema".
Repositorio	Esconde la base de datos. Ni el ViewModel ni la pantalla tocan Room directamente.	La "cocina": nadie de afuera entra, solo le piden platos por una ventanilla.
Room	Librería que guarda los datos en una base SQLite dentro del celular. Persistente: sobrevive a cerrar la app.	Un cuaderno físico en vez de una nota en un papelito que se pierde.
AppContainer	Clase chiquita que crea la base de datos y los repositorios **una sola vez** y los presta a quien los necesite.	La bodega central: nadie compra sus propios insumos por su cuenta.
Route vs Screen	`XRoute` es la parte que sí conoce al ViewModel; `XScreen` solo dibuja según lo que le pasan, nunca ve el ViewModel.	`Route` es el mesero que va y viene; `Screen` es el mantel y los platos ya servidos en la mesa.
Tabla maestra	Guarda catálogos o entidades principales que casi no cambian.	Como la lista de sucursales de un banco: hay pocas y casi no cambian.
Tabla transaccional	Guarda "eventos" que van pasando en el tiempo (cada cuidado registrado es una fila nueva).	Como el historial de movimientos de una cuenta bancaria: cada movimiento es una fila nueva.
Channel (eventos puntuales)	Un "mensajero" que el ViewModel usa para avisarle a la pantalla algo que pasa una sola vez (ej. "se guardó bien", "hubo un error").	Como una notificación push: aparece una vez y desaparece, no se queda pegada como el fondo de la pantalla.

Las 4 tareas

Tarea	Persona	Archivos	Qué hace
1 — Home	■	`HomeScreen.kt` + `HomeViewModel.kt`	Resumen de cuántas mascotas hay.

Tarea	Persona	Archivos	Qué hace
2 — Mis Mascotas	■	`MascotasScreen.kt` + `MascotasViewModel.kt`	Lista de mascotas.
3 — Detalle + Cuidados	■	`DetalleMascotaScreen.kt` + `DetalleMascotaViewModel.kt` **más** `RegistrarCuidadoScreen.kt` + `RegistrarCuidadoViewModel.kt`	Detalle muestra los datos de la mascota y su lista de cuidados, con un botón "Registrar cuidado". La pantalla nueva sirve para crear **y** editar un cuidado (según si le llega un id), con un formulario: tipo de cuidado (menú desplegable), fecha, notas — y un botón para eliminar si está editando.
4 — Datos + contenedor	■	`PetCareApp.kt`, todo `data/` (3 tablas), `AppContainer.kt`	Arma las 3 tablas de Room, los 2 repositorios (`MascotaRepository`, `CuidadoRepository`) y decide qué pantalla mostrar con la variable de estado.

Aviso: la Tarea 3 es la más grande (2 pantallas). Si Home o Mascotas se terminan rápido, esa persona puede ayudar con `RegistrarCuidadoScreen`. La Tarea 4 también se puede repartir entre 2 personas en paralelo (ver el orden de abajo).

Antes de empezar: el orden correcto para no bloquearse entre sí

1. **Gradle**: dependencias de Room + ViewModel de Compose (no depende de nada, se puede hacer ya).
2. `Pantalla.kt`: el enum con las 4 pantallas (`HOME`, `MASCOTAS`, `DETALLE`, `REGISTRAR_CUIDADO`) — tampoco depende de nada.
3. **El bloque que sí bloquea a todos**: las 3 entidades (`Mascota.kt`, `TipoCuidado.kt`, `Cuidado.kt`) + las 2 interfaces de repositorio (`MascotaRepository.kt`, `CuidadoRepository.kt`). Van juntas porque `Cuidado` necesita que `Mascota` y `TipoCuidado` ya existan (las usa como llave foránea), y `CuidadoRepository` usa los 3 tipos en su firma. En cuanto esto sube, **las tareas 1, 2 y 3 ya pueden arrancar sus ViewModels**, y los 2 sub-hilos de abajo también pueden arrancar en paralelo.
4. **En paralelo**: Sub-hilo A (Dao + implementación de `Mascota`) y Sub-hilo B (Dao + implementación de `TipoCuidado`/`Cuidado`) — cada uno construye sobre las entidades del punto 3, no dependen entre sí.
5. **Al final**: quien lidere la Tarea 4 junta ambos sub-hilos en `PetCareDatabase.kt` y `AppContainer.kt`.

Cómo lo probamos entre todos al final

1. Cada quien sube su(s) archivo(s) en su propia rama de Git y hace su Pull Request.
2. Se integra en orden: `data/` completo → pantallas → `PetCareApp.kt`.
3. Prueba completa: abrir la app → ir a Detalle de Rocky → registrar un cuidado (elegir tipo, poner fecha y notas) → verificar que aparece en la lista al instante → editarlo → eliminarlo → **cerrar la app por completo, poner modo avión, volver a abrirla** → todo lo que no borraste debe seguir ahí.
4. Revisar que ninguna pantalla (`Screen`) reciba el ViewModel directamente — solo el estado ya resuelto y lambdas.

Recordatorio: presentación de cierre (5 minutos)

- **1 min** — nombre de la app y el problema que resuelve.
- **2 min** — demo en vivo: registrar/editar/eliminar un cuidado de Rocky, y la prueba de offline (cerrar app + modo avión + reabrir).
- **2 min** — arquitectura: mostrar `Route`/`Screen` sin pasar el ViewModel, el flujo de datos desde Room hasta la UI, y cómo se entregan el repositorio y el ViewModel sin instanciarlos a mano.
- Los 4 integrantes deben poder responder preguntas de cualquier capa, así que conviene que cada quien entienda al menos su propia capa a fondo.

Qué sí entra y qué no en esta entrega

✅ Sí entra	❌ Todavía no
3 tablas en Room (2 maestras + 1 transaccional)	NavHost / NavController
CRUD completo de `Cuidado` (crear, ver, editar, eliminar)	Registrar/editar/eliminar `Mascota` (sigue solo con Rocky precargado)
Eventos puntuales con `Channel` (avisos de "guardado"/"error")	DataStore, Splash Screen, pantalla de Configuración
Componentes Material 3 en todas las pantallas	Hilt (usamos `AppContainer` a mano)
ViewModel + `StateFlow<UiState>` por pantalla	UseCases, ícono adaptativo, `WindowSizeClass` (son extras opcionales)
Patrón Repositorio (2 repositorios)	Que el botón físico de "atrás" navegue solo entre pantallas

---

Documento generado a partir del plan técnico completo (ver `plan-tecnico-implementacion.pdf` en esta misma carpeta) para que sea más fácil de repartir y leer en el chat del grupo.